1 Kotlin Programming Language

Introduction to Kotlin

Contents

1 **Introduction to Kotlin**

History, philosophy, why Kotlin, and comparison with Java

2 **Core Syntax & Fundamentals**

Variables, types, control flow, functions, and null safety

3 **OOP & Collections**

Classes, data classes, objects, lists, sets, maps, and functional operations

4 **Advanced Features & Ecosystem**

Coroutines, multiplatform, tooling, and learning resources

What is Kotlin?

Kotlin is a modern, statically typed programming language developed by JetBrains, first released in 2016. It runs on the JVM, JavaScript, and Native platforms.

Key Facts

Created by JetBrains

Developed by JetBrains (makers of IntelliJ IDEA). Open-sourced under the Apache 2.0 license.

Google Endorsed

In 2017, Google announced first-class support for Kotlin on Android. Now the preferred language.

Multi-paradigm

Supports object-oriented and functional programming styles. Combines the best of both worlds.

100% Java Interop

Fully interoperable with Java. You can call Java code from Kotlin and vice versa seamlessly.

Why Kotlin?

1

Concise

Drastically reduces boilerplate code. Data classes, type inference, and smart casts save time and lines of code.

2

Safe

Built-in null safety eliminates NullPointerExceptions at compile time. The type system distinguishes nullable and non-null types.

3

Interoperable

100% interoperable with Java and the JVM ecosystem. Use any Java library, framework, or tool directly in Kotlin.

4

Multiplatform

Share code across Android, iOS, web, desktop, and server. Write business logic once, deploy everywhere.

5

Tool-Friendly

First-class IDE support in IntelliJ IDEA and Android Studio. Excellent debugging, refactoring, and code completion.

Kotlin vs Java

Kotlin	Java
Null safety built into the type system	Null checks are manual, NPEs are common
Data classes with one line of code	Verbose POJOs: getters, setters, equals, hashCode
Extension functions add behaviour	Utility classes with static methods
Coroutines for async programming	Threads and callbacks, or CompletableFuture
Smart casts eliminate explicit casting	Manual instanceof checks and explicit casts
String templates: "Hello, $name"	String concatenation or String.format()
val/var for immutable/mutable variables	final keyword, mutable by default
When expressions replace switch	Switch statements with break required

Basic Syntax

Variables & Types

```
val name: String = "Kotlin"
var age: Int = 10
val pi = 3.14  // Type inferred
val isActive = true
val items: List = listOf(1, 2)
```

val = immutable (read-only, like final)
var = mutable (can be reassigned)
Type inference: compiler deduces types
Basic types: Int, Double, Boolean, String, Char, Long, Float, Short, Byte

Entry Point & Output

```
fun main() {
    val greeting = "Hello, World!"
    println(greeting)
}
```

fun main() is the program entry point
No class wrapper needed (unlike Java)
Semicolons are optional
println() prints to standard output

Control Flow

if / when Expressions

```
// if is an expression
val max = if (a > b) a else b

// when replaces switch
when (x) {
    1 -> "One"
    else -> "Other"
}
```

if is an expression that returns a value
when replaces switch, is exhaustive
No ternary operator: use if expression
Can match ranges, types, conditions

for / while Loops

```
for (i in 1..5) { print(i) }

for (i in 10 downTo 0 step 2) {
    println(i)
}
```

for iterates over anything with an iterator
Ranges: 1..5 (inclusive), 1 until 5 (exclusive)
downTo for reverse, step for increments
while and do-while also supported

Functions

```
// Basic function
fun greet(name: String): String {
    return "Hello, $name!"
}

// Single-expression function
fun double(x: Int) = x * 2

// Default parameters
fun log(msg: String,
        level: String = "INFO")

// Lambda expression
val sum = { a: Int, b: Int -> a + b }
```

Function Basics

- Declared with fun keyword
- Return type after colon (:)
- Unit (void) return type can be omitted

Key Features

- Single-expression = concise syntax
- Default params reduce overloads
- Named arguments for clarity
- Lambdas are first-class citizens
- Extension functions add methods

Null Safety

Kotlin's type system distinguishes nullable (String?) and non-null (String) references, eliminating NullPointerExceptions at compile time.

```kotlin
// Nullable type
var name: String? = "Kotlin"
name = null  // OK

// Safe call operator
val len = name?.length

// Elvis operator
val l = name?.length ?: 0

// Not-null assertion
val l2 = name!!.length
```

Nullable Types (T?)

- Add ? to allow null values
- Compiler enforces null checks

Safe Call (?.)

- Returns null if receiver is null
- Chainable: a?.b?.c

Elvis Operator (?:)

- Provides default for null values

Not-null Assertion (!!)

- Throws NPE if null — use sparingly!

Classes & Objects

Kotlin classes are concise by default — no new keyword needed, properties built into constructors.

class

```
class Person(
  val name: String,
```

Primary constructor in header

data class

```
data class User(
  val name: String,
```

Auto equals, hashCode, copy, toString

object

```
object Database {
  val url = "jdbc:
```

Singleton pattern built-in

companion

```
class MyClass {
companion object {
```

Like Java static members

Additional Class Features

- No **new** keyword needed to create instances
- Classes are **final by default** — use **open** to allow inheritance
- Sealed classes restrict inheritance hierarchies (great for **when** expressions)
- Interfaces can contain default method implementations and abstract properties

Collections

Creating Collections

```
val list = listOf(1, 2, 3)
val mutableList = mutableListOf()
val set = setOf("a", "b")
val map = mapOf(1 to "one")
```

Key Concepts

- Immutable by default (List, Set, Map)
- Mutable variants: MutableList, etc.
- Rich functional API included
- Type-safe builders available

Functional Operations

```
val numbers = listOf(1, 2, 3, 4, 5, 6, 7, 8)

val evens = numbers.filter { it % 2 == 0 }      // [2, 4, 6, 8]
val doubled = numbers.map { it * 2 }            // [2, 4, 6, ...]
val sum = numbers.reduce { acc, n -> acc + n }  // 36
val first3 = numbers.take(3)                    // [1, 2, 3]
val grouped = numbers.groupBy { it % 2 }        // {1=[1,3,5,7], 0=[2,4,6,8]}
```

Coroutines

Kotlin coroutines provide lightweight, structured concurrency for asynchronous programming without callbacks or complex threading.

```
import kotlinx.coroutines.*

// Suspend function
suspend fun fetchData(): String {
    delay(1000)
    return "Result"
}

// Launch a coroutine
fun main() = runBlocking {
    val result = async {
        fetchData()
    }
    println(result.await())
}
```

Key Concepts

- **suspend** marks pausable functions
- **launch** starts fire-and-forget jobs
- **async** returns Deferred results
- **runBlocking** bridges blocking code

Dispatchers

- Main — UI thread
- IO — network/disk operations
- Default — CPU-intensive work

Kotlin Multiplatform

Write shared business logic once, deploy to Android, iOS, web, desktop, and server. Platform-specific code only where needed.

Target Platforms

JVM / Android

Share code with Android apps. Full Compose Multiplatform support.

iOS / Native

Compile to native binaries. Share logic with Swift/SwiftUI frontends.

JavaScript / Web

Compile to JS for browser or Node.js. Use with React or Kotlin/JS.

WASM

Kotlin/Wasm for WebAssembly targets. Near-native performance in browsers.

How KMP Works

- Common module contains shared business logic, data models, and networking
- Use `expect/actual` declarations for platform-specific implementations
- Compose Multiplatform enables shared UI across Android, iOS, desktop, and web

Ecosystem & Tools

1 IDE Support

IntelliJ IDEA and Android Studio provide first-class Kotlin support with refactoring, debugging, and code completion.

2 Build Tools

Gradle with Kotlin DSL (build.gradle.kts) is the standard. Maven support also available. Version catalogs for deps.

3 Key Libraries

Ktor (HTTP), Exposed (DB), Serialization (JSON), Koin/Hilt (DI), Arrow (FP), Compose (UI).

4 Testing

kotlin.test for cross-platform tests, MockK for mocking, Kotest for property-based testing, Turbine for flows.

Summary & Resources

Key Takeaways

- Modern, concise, and safe language
- Built-in null safety prevents crashes
- 100% interoperable with Java
- Coroutines simplify async code
- Multiplatform enables code sharing
- Preferred language for Android
- Rich ecosystem of libraries and tools
- Strong community and JetBrains backing

Learning Resources

Official

- kotlinlang.org — docs and tutorials
- play.kotlinlang.org — online sandbox
- Kotlin Koans — interactive exercises

Community

- Kotlin Slack — active community
- KotlinConf — annual conference
- Kotlin Weekly — newsletter